

the company and the department they belong to. If you are on the operations side of the fence, then DevOps could mean applying SDLC (software development life cycle) techniques to your project management in order to be able to collaborate better.

On the other hand, if you are a developer, it could mean utilising tools and mechanisms that leverage the principles of lean and/or agile in order to collaborate with operations and ensure a smooth rollout and maintenance of the product.

The circumference of influence of DevOps is far reaching. Even a client, who is the recipient of the product, is a player in the DevOps framework.

Therefore, though there are many textbooks out there that have done a decent enough job of defining DevOps, the answer to the question ultimately is that there is no standard definition of DevOps. It can only be defined in rudimentary terms that act more as an overview to its underlying intricacies.

So with that in mind, I shall attempt, for the sake of this article, to provide you with a non-granular, elementary definition of DevOps.

“DevOps is a term used to describe the most optimal practices, tools and frameworks used to collaborate between all parties involved in the software development process, all the way from inception to delivery and beyond.”

That’s my take on DevOps. Feel free to come up with your own. Whatever sails your boat.

## Open Source and DevOps

As a developer, the most common terms that come to mind when I think of DevOps are terms such as Git, Docker, Jenkins, etc.

What do they all have in common? Two things. First, they are all what you could define as DevOps tools. Second, most (if not all) are open source. There is a deep and inherent connection between DevOps and open source.

And there is a deep and inherent reason for that as well. If you think about it, DevOps is nothing but a means of collaboration between different players working on different aspects of the product development and deployment.

But collaboration in software development looks vastly different today from what it looked like in the early 2000s. Today, software is built, not by people working together in the same location, but by people collaborating across the globe. An overwhelming amount of software projects today are open source. Even big tech has come to kneel before FOSS. Why? It’s more efficient, fast, secure and, simply put, cheap. Tech companies have everything to gain and little to lose when it comes to open source DevOps tools.

With this comes the challenge of making the software work on multiple devices and operating systems with varying requirements and dependencies. There is also the fact that not everyone contributes code. Contributions to the software come in many forms, including documentation, tutorials, maintenance, bug reports, testing, security updates, and much more. It is a complex process that requires tools that can handle complex frameworks. This has created an inherent need within the software industry to create open source DevOps tools.

## Current trends

Current trends are clear. DevOps tools are king. If you want to be a DevOps engineer or want to do anything remotely connected to DevOps, you are, by default, expected to be experienced in DevOps tools. As discussed in the previous sections, the definition of DevOps far exceeds that of a few tools used for automation and collaboration. However, there has been a noticeable surge in the demand for open source tools such as Docker, Jenkins, Kubernetes and Git.

According to the 2020 report released by Burning Glass, which collects and analyses millions of job postings every year, the primary skills required to be a DevOps engineer are (in order of importance):

- DevOps tools (Kubernetes, Docker, Git, Jenkins, etc)
- Python
- Linux
- Ansible
- Java
- Infrastructure automation

It comes as no surprise that DevOps tools make it to the top of the list. Additionally, let us have a look at the Google search trends for the past five years for popular DevOps tools.

Figure 1 and Figure 2 show a clear rise in Google searches for Kubernetes and Docker. Figure 3 and Figure 4 show that more aged technologies such as Git and Jenkins have remained popular throughout with no visible decline in public interest.

It becomes abundantly clear from these trends that though DevOps is a combination of soft and hard skills, the latter clearly take an upper hand in the equation to the extent that the term DevOps has now essentially become synonymous with FOSS based DevOps tools.

## Future of DevOps

Before I start making predictions about DevOps, I feel compelled to make it clear that I am not a DevOps engineer. And while DevOps plays a huge role in my job, I am in no way an expert in it.

The reason I chose to write this article is that as an engineer who specialises in artificial intelligence, automation is ingrained into my very spirit. Automation of labour intensive tasks that reduce unnecessary bureaucracy and unrequired paperwork is the very principle on which AI functions. As such, it only makes sense that my field of expertise collides with DevOps.